Examination System

Backend-Focused User Stories & Acceptance Criteria

For Backend Engineering Team

📦 Epic 1: Authentication & Identity

User Story 1.1 — User Registration

Endpoint: POST /api/auth/register
Role: Public
Description: Register a new user via email and password.

Request Payload

```
{ email, password, full_name }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid email + password provided	POST /api/auth/register	Create user record, hash password (bcrypt), return 201 with user ID. Send OTP to email.
Email already exists in DB	POST /api/auth/register	Return 409 Conflict with message 'Email already registered'.
Missing required fields	POST /api/auth/register	Return 422 Unprocessable Entity with field-level validation errors.
Password < 8 chars or no uppercase/special char	POST /api/auth/register	Return 422 with specific password policy violation message.

📌 *Backend Notes: Store hashed passwords only (bcrypt, salt rounds ≥ 12). Never log raw passwords. Set account status = 'pending' until email verified.*

User Story 1.2 — User Login

Endpoint: POST /api/auth/login
Role: Public
Description: Authenticate user and return JWT access + refresh tokens.

Request Payload

```
{ email, password }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid credentials, account is active + verified	POST /api/auth/login	Return 200 with access_token (JWT, 15min expiry) and refresh_token (httpOnly cookie, 7d expiry).
Invalid password	POST /api/auth/login	Return 401 Unauthorized. Increment failed attempt counter. Do not reveal which field is wrong.

Given	When	Then (Expected Backend Behavior)
Account not verified	POST /api/auth/login	Return 403 Forbidden with message 'Account not verified'.
5 consecutive failed attempts	POST /api/auth/login	Lock account for 15 minutes. Return 429 Too Many Requests.

📌 *Backend Notes: JWT payload must include: user_id, role, exp. Use RS256 or HS256. Refresh token rotation on every use. Log login attempts with IP and timestamp.*

User Story 1.3 — Password Reset

Endpoints: POST /api/auth/forgot-password | POST /api/auth/reset-password
Role: Public

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid registered email submitted	POST /api/auth/forgot-password	Generate a signed reset token (UUID or JWT), store with 15-min TTL, send link to email. Return 200 regardless of email existence (prevent enumeration).
Valid token + new password submitted	POST /api/auth/reset-password	Validate token (not expired, not used), hash new password, update DB record, invalidate token. Return 200.
Expired or already-used token	POST /api/auth/reset-password	Return 400 Bad Request with message 'Token invalid or expired'.

📌 *Backend Notes: Tokens must be single-use. Invalidate all active sessions (refresh tokens) upon successful password reset. Store token hash in DB, not plain token.*

User Story 1.4 — Account Verification (OTP)

Endpoints: POST /api/auth/verify-otp | POST /api/auth/resend-otp
Role: Unauthenticated (pending user)

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Correct OTP submitted within expiry window	POST /api/auth/verify-otp	Set account status = 'active'. Return 200. Invalidate OTP record.
Incorrect OTP submitted	POST /api/auth/verify-otp	Return 400. Increment attempt counter. Lock after 5 attempts.
OTP expired (> 10 minutes)	POST /api/auth/verify-otp	Return 400 with 'OTP expired'. Do not activate account.
Valid email, resend requested	POST /api/auth/resend-otp	Invalidate old OTP, generate new 6-digit OTP, set 10-min TTL, resend email. Limit to 3 resends per hour.

📌 *Backend Notes: OTP must be 6 digits, cryptographically random. Store hashed OTP in DB. Rate-limit resend endpoint per IP and per email.*

📦 Epic 2: Student Dashboard & Learning

User Story 2.1 — View Dashboard

Endpoint: GET /api/student/dashboard
Role: Authenticated Student

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Authenticated student sends request	GET /api/student/dashboard	Return 200 with: enrolled_diplomas[], recent_quiz_attempts[], overall_stats { total_quizzes_taken, avg_score, pass_rate }.
JWT missing or invalid	GET /api/student/dashboard	Return 401 Unauthorized.
User role is not 'student'	GET /api/student/dashboard	Return 403 Forbidden.

📌 *Backend Notes: Aggregate stats from quiz_attempts table. Eager-load diploma progress in a single optimized query. Cache dashboard response for 60 seconds per user_id.*

User Story 2.2 — View Diplomas

Endpoint: GET /api/diplomas
Role: Authenticated Student

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Diplomas exist and are published	GET /api/diplomas	Return 200 with paginated array: [{ id, title, description, quiz_count, student_progress }].
No published diplomas exist	GET /api/diplomas	Return 200 with empty array []. Do not return 404.

📌 *Backend Notes: Only return diplomas with status = 'published'. Include pagination meta (page, per_page, total). Filter by student's enrollment where applicable.*

User Story 2.3 — Select Diploma / View Quizzes

Endpoint: GET /api/diplomas/:diploma_id/quizzes
Role: Authenticated Student

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid diploma_id, diploma is published	GET /api/diplomas/:id/quizzes	Return 200 with quizzes array: [{ id, title, duration_minutes, attempt_count, last_score, status }].
diploma_id not found or not published	GET /api/diplomas/:id/quizzes	Return 404 Not Found.
Student not enrolled in diploma	GET /api/diplomas/:id/quizzes	Return 403 Forbidden (if enrollment is required).

📌 *Backend Notes: Join with quiz_attempts to attach per-student attempt history. Only return quizzes with status = 'published'.*

📦 Epic 3: Quiz Engine

User Story 3.1 — Start Quiz

Endpoint: POST /api/quizzes/:quiz_id/start
Role: Authenticated Student

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Quiz is published and student hasn't exceeded attempt limit	POST /api/quizzes/:id/start	Create attempt record (status='in_progress'), record start_time, return attempt_id, quiz metadata, and shuffled questions (without revealing correct answers).
Quiz not found or not published	POST /api/quizzes/:id/start	Return 404 Not Found.
Student has reached max attempt limit	POST /api/quizzes/:id/start	Return 403 with message 'Attempt limit reached'.
Student already has an in-progress attempt for this quiz	POST /api/quizzes/:id/start	Return 409 Conflict. Return the existing attempt_id for resumption.

📌 *Backend Notes: NEVER include correct_answer or answer_explanation in the start response. Store attempt server-side keyed to attempt_id. Shuffle question order and option order per attempt.*

User Story 3.2 — Answer Questions

Endpoint: POST /api/attempts/:attempt_id/answer
Role: Authenticated Student (attempt owner)

Request Payload

```
{ question_id, selected_option_id }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid attempt_id (in_progress) + valid question_id + valid option_id	POST /api/attempts/:id/answer	Upsert answer in attempt_answers table. Return 200 with { saved: true }.
Attempt is already submitted or expired	POST /api/attempts/:id/answer	Return 409 Conflict. Do not update answers.
Student does not own this attempt	POST /api/attempts/:id/answer	Return 403 Forbidden.
question_id not part of this quiz	POST /api/attempts/:id/answer	Return 422 Unprocessable Entity.
Timer has elapsed on server side	POST /api/attempts/:id/answer	Return 410 Gone. Trigger auto-submit.

📌 *Backend Notes: Always validate attempt ownership server-side. Use UPSERT so re-answering overwrites the previous answer. Do not trust client-side timer — validate elapsed time on every answer submission.*

User Story 3.3 — Timer Handling

Mechanism: Server-side timer enforcement (not client-only)

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Attempt is started	Quiz timer begins	Record start_time in DB. Calculate deadline = start_time + duration_minutes.
Any API request arrives for an attempt past its deadline	(Any quiz endpoint)	Trigger auto-submit: mark attempt status='timed_out', calculate score from answers submitted so far, return 410 Gone.
Client polls for remaining time	GET /api/attempts/:id/timer	Return { seconds_remaining }. Calculate server-side: deadline - now().

📌 *Backend Notes: NEVER rely solely on client-side timer. Enforce deadline on every attempt-related request. A background job or middleware check is acceptable, but server-side validation on each request is mandatory.*

User Story 3.4 — Submit Quiz

Endpoint: POST /api/attempts/:attempt_id/submit
Role: Authenticated Student (attempt owner)

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Attempt is in_progress, student owns it	POST /api/attempts/:id/submit	Set status='submitted', set submitted_at = now(). Trigger score calculation. Return 200 with { attempt_id, score, passed }.
Attempt already submitted	POST /api/attempts/:id/submit	Return 409 Conflict. Return existing result.

Given	When	Then (Expected Backend Behavior)
Timer has expired before explicit submit	POST /api/attempts/:id/submit	Auto-submit with answers on record. Set status='timed_out'. Score based on answers submitted before deadline.

📌 *Backend Notes: Score calculation: (correct_answers / total_questions) × 100. Passing threshold is configurable per quiz (pass_score field). Store score, pass/fail, and per-question breakdown in attempt_results table.*

User Story 3.5 — View Results

Endpoint: GET /api/attempts/:attempt_id/results
Role: Authenticated Student (attempt owner) or Admin

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Attempt is submitted/timed_out, requester is owner or admin	GET /api/attempts/:id/results	Return 200 with: { score, passed, total_questions, correct_count, per_question: [{ question_id, student_answer, correct_answer, is_correct }] }.
Attempt is still in_progress	GET /api/attempts/:id/results	Return 403 — results not available until submitted.
Requester does not own attempt and is not admin	GET /api/attempts/:id/results	Return 403 Forbidden.

📌 *Backend Notes: correct_answer field must ONLY be returned after submission. Include explanation field per question if available. Do not expose other students' data.*

User Story 3.6 — Quiz History

Endpoint: GET /api/student/attempts
Role: Authenticated Student

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Student has past attempts	GET /api/student/attempts	Return paginated list: [{ attempt_id, quiz_title, score, passed, status, submitted_at }]. Sort by submitted_at DESC.
No attempts exist	GET /api/student/attempts	Return 200 with empty array [].
Attempt ID passed as filter	GET /api/student/attempts/:attempt_id	Return full result detail for that specific attempt.

📌 *Backend Notes: Support query params: ?quiz_id=, ?diploma_id=, ?page=, ?per_page=. Only return the authenticated student's own attempts.*

📦 Epic 4: Admin Management

User Story 4.1 — Admin Dashboard Stats

Endpoint: GET /api/admin/stats
Role: Admin

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Admin is authenticated	GET /api/admin/stats	Return 200 with: { total_users, active_users_today, total_quizzes, total_attempts, avg_pass_rate }.
Non-admin role accesses endpoint	GET /api/admin/stats	Return 403 Forbidden.

> 📌 *Backend Notes: Cache stats response for 5 minutes. Use database aggregation queries, not in-memory counting. Separate active_users_today with a session/login log query.*

User Story 4.2 — Manage Diplomas (CRUD)

Endpoints: POST /api/admin/diplomas | PUT /api/admin/diplomas/:id | DELETE /api/admin/diplomas/:id
Role: Admin

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid diploma payload submitted	POST /api/admin/diplomas	Create diploma with status='draft'. Return 201 with diploma object.
Invalid or missing required fields (title)	POST /api/admin/diplomas	Return 422 with field-level validation errors.
Valid diploma_id + update payload	PUT /api/admin/diplomas/:id	Update fields. Return 200 with updated diploma. Disallow changing status via this endpoint.
Diploma has active student enrollments	DELETE /api/admin/diplomas/:id	Return 409 Conflict. Soft-delete only (set deleted_at). Do not hard-delete.

> 📌 *Backend Notes: Use soft delete (deleted_at timestamp). Diplomas with status='published' and active enrollments cannot be deleted. Changing published status is handled by a dedicated publish endpoint.*

User Story 4.3 — Create & Manage Quizzes

Endpoints: POST /api/admin/quizzes | PUT /api/admin/quizzes/:id
Role: Admin

Request Payload

```
{ title, diploma_id, duration_minutes, pass_score, max_attempts, instructions }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid quiz payload submitted	POST /api/admin/quizzes	Create quiz with status='draft'. Return 201 with quiz_id and full object.
diploma_id does not exist	POST /api/admin/quizzes	Return 404 Not Found.
Missing required fields (title, duration_minutes, pass_score)	POST /api/admin/quizzes	Return 422 with field-level validation errors.
pass_score not between 0–100	POST /api/admin/quizzes	Return 422 with validation message.

📌 *Backend Notes: Quiz must have at least 1 question before it can be published. duration_minutes must be a positive integer. pass_score default = 60 if not provided.*

User Story 4.4 — Manage Questions & Options

Endpoints: POST /api/admin/quizzes/:quiz_id/questions | PUT /api/admin/questions/:id | DELETE /api/admin/questions/:id
Role: Admin

Request Payload

```
{ text, options: [{ text, is_correct }], explanation }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid question with at least 2 options and exactly 1 correct	POST /api/admin/quizzes/:id/questions	Create question + options. Return 201 with question_id.
No correct answer marked or more than 1 marked	POST /api/admin/quizzes/:id/questions	Return 422 with message 'Exactly one correct option required'.
Fewer than 2 options provided	POST /api/admin/quizzes/:id/questions	Return 422 with validation error.
Question deleted while quiz is published	DELETE /api/admin/questions/:id	Return 409. Unpublish quiz first or soft-delete question.

📌 *Backend Notes: Store correct answer separately or with an is_correct flag. Never expose is_correct to students until after submission. Support question ordering (order_index field).*

User Story 4.5 — Publish Quiz

Endpoint: PATCH /api/admin/quizzes/:quiz_id/publish
Role: Admin

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Quiz has ≥ 1 question and all questions have valid options	PATCH /api/admin/quizzes/:id/publish	Set status='published'. Return 200 with { quiz_id, status: 'published' }.
Quiz has 0 questions	PATCH /api/admin/quizzes/:id/publish	Return 422 with message 'Quiz must have at least one question'.
Quiz is already published	PATCH /api/admin/quizzes/:id/publish	Return 409 Conflict.
Admin wants to unpublish	PATCH /api/admin/quizzes/:id/unpublish	Only allowed if no in-progress attempts exist. Return 409 if active attempts found.

📌 *Backend Notes: Publish is a state transition — validate all business rules before allowing it. Consider a pre-publish validation endpoint (GET /api/admin/quizzes/:id/validate) that returns a readiness checklist.*

📦 Epic 5: Monitoring & Analytics

User Story 5.1 — View Student Attempts (Admin)

Endpoint: GET /api/admin/attempts
Role: Admin

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Admin requests all attempts	GET /api/admin/attempts	Return paginated list: [{ attempt_id, student_id, student_name, quiz_title, score, status, submitted_at }].
Filter by quiz_id or student_id applied	GET /api/admin/attempts?quiz_id=&student_id=	Return filtered results matching both params. Support combining filters.
Specific attempt detail requested	GET /api/admin/attempts/:attempt_id	Return full attempt detail including per-question breakdown.
Non-admin requests this endpoint	GET /api/admin/attempts	Return 403 Forbidden.

📌 *Backend Notes: Paginate all list endpoints (default page=1, per_page=20). Support sorting: ?sort_by=submitted_at&order=desc. Index quiz_id and student_id columns for performance.*

User Story 5.2 — Performance Analytics

Endpoint: GET /api/admin/analytics
Role: Admin

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Admin requests analytics with no filters	GET /api/admin/analytics	Return: { pass_rate_by_quiz[], avg_score_by_diploma[], attempts_over_time[], top_failed_questions[] }.
Filter by date range applied	GET /api/admin/analytics?from=&to=	Return analytics scoped to that date range only.
Filter by diploma_id applied	GET /api/admin/analytics?diploma_id=	Scope all metrics to that diploma's quizzes.
No data matches filter	GET /api/admin/analytics	Return 200 with empty arrays/zeroes. Do not return 404.

📌 *Backend Notes: Analytics queries can be expensive — use materialized views or cache results for 10 minutes. top_failed_questions = questions with correct_answer rate < 40%. All timestamps should be returned in UTC ISO 8601 format.*

⚙️ General Backend Requirements

Given	When	Then (Expected Backend Behavior)
All endpoints	Any request	Validate JWT in Authorization header (Bearer scheme). Return 401 if missing or invalid.
All write endpoints	POST / PUT / PATCH / DELETE	Require Content-Type: application/json. Return 415 if not set.
Any input field	Any mutation endpoint	Sanitize all inputs. Never pass raw user input to queries (use parameterized queries / ORM).
All paginated endpoints	GET list endpoints	Support ?page= and ?per_page= (max 100). Return meta: { page, per_page, total, total_pages }.
All responses	Any endpoint	Return consistent JSON envelope: { success, data, error, meta }.
Rate limiting	Auth endpoints	Limit: 10 req/min on /login, /register, /verify-otp. Return 429 with Retry-After header.